PRACTICAL FILE

Advanced DBMS and Administration

(Paper 1T4)

First Year M.C.A. — Semester I (CBCS)

RTM Nagpur University

Name: _____

Roll No: _____

INDEX

Sr. No.	Practical	Date	Sign
1	Create a new user & grant privileges of creating session, table & view to this new user also check connect or not.		
2	Connect to the new user, assign a tablespace, verify it works, create a table, and insert data.		
3	Create a new role named manager with same privileges. Grant this role to new user. Then change the password of newly created user.		
4	Revoke the privileges from role manager. Revoke role from user then delete that user.		
5	Create a tablespace named “P_student” with file name “Stu_datafile.dbf” having size of 10mb.		
6	Create a tablespace named “ts_mydemo” with two datafile named “ts_mydemo01.dbf” & “ts_mydemo02.dbf” having size of 50mb and 64mb respectively. Make the tablespace auto extendable and limit auto extending to size of 2048mb. In auto extending, next extend should of 32mb.		
7	Alter the last datafile “ts_mydemo01.dbf” so that its size will automatically extend by 5mb & limit its size to 500mb.		
8	Create temporary table space with size of 5mb which will be automatically increase on demand.		
9	Add new Redo log file of size 10mb to existing database.		
10	Display the list of all Redo log files and Control files.		
11	Display the list, datafiles, tablespace names & size in MB.		
12	Create rollback segment in separate tablespace with proper storage option.		
13	Create a 3 new abstract datatype as given below: 1. NAME_TYPE(F_NAME, M_NAME, L_NAME) 2. POST_TYPE(MAIN, SFX) 3. ADDR_TYPE(STREET, CITY, STATE, POSTCODE)		
14	Create a table stud_info having column name, rollno, phy, maths, comp, total and percentage. Write a trigger to automatically calculate students total marks and percentage of students while insertion.		
15	Write a SQL query to display the following data information: a. Name of database b. Database startup time c. Database size information (database size, used space, free space)		

Practical No. 1

Aim: Create a new user & grant privileges of creating session, table & view to this new user also check connect or not.

Step 1: Login as DBA (SYSTEM user)

Description: Only a DBA user has the rights to create users, tablespaces and change the database, so first connect as SYSTEM in SQL*Plus.

Credentials: Username: system, Password: the password set during Oracle installation.

Syntax:

```
CONNECT username/password;
```

Example:

```
SQL> connect system/<password>;
Connected.
```

Step 2: Create a new user

Description: CREATE USER creates a new database user (schema) with a password.

Syntax:

```
CREATE USER username IDENTIFIED BY password;
```

Example:

```
SQL> create user Prerna identified by prerna123;

User created.
```

Step 3: Grant privileges to the user

Description: GRANT gives system privileges: CREATE SESSION (to login), CREATE TABLE and CREATE VIEW.

Syntax:

```
GRANT privilege1, privilege2, ... TO username;
```

Example:

```
SQL> grant create table, create session, create view to Prerna;

Grant succeeded.
```

Step 4: Check the connection with the new user

Description: Connect as the new user. The message "Connected." confirms that the user and the CREATE SESSION privilege work.

Credentials: Username: Prerna, Password: prerna123

Syntax:

```
CONNECT username/password;
```

Example:

```
SQL> CONNECT Prerna/prerna123;

Connected.
```

Practical No. 2

Aim: Connect to the new user, assign a tablespace, verify it works, create a table, and insert data.

Step 1: Login as DBA (SYSTEM user)

Description: Only a DBA user has the rights to create users, tablespaces and change the database, so first connect as SYSTEM in SQL*Plus.

Credentials: Username: system, Password: the password set during Oracle installation.

Syntax:

```
CONNECT username/password;
```

Example:

```
SQL> connect system/<password>;
Connected.
```

Step 2: Assign tablespace and quota to the user

Description: ALTER USER sets the default tablespace (where the user's tables are stored), the temporary tablespace and the quota (space allowed).

Syntax:

```
ALTER USER username
    DEFAULT TABLESPACE tablespace_name
    TEMPORARY TABLESPACE temp_tablespace
    QUOTA size ON tablespace_name;
```

Example:

```
SQL> ALTER USER Prerna
  2      DEFAULT TABLESPACE users
  3      TEMPORARY TABLESPACE temp
  4      QUOTA 10M ON users;

User altered.
```

Step 3: Connect as the new user

Description: Login with the new user to work in its own schema.

Credentials: Username: Prerna, Password: prerna123

Syntax:

```
CONN username/password;
```

Example:

```
SQL> conn
Enter user-name: Prerna
Enter password:
Connected.
```

Step 4: Create a table

Description: CREATE TABLE creates the table in the user's default tablespace (users).

Syntax:

```
CREATE TABLE table_name (
    column1 datatype [constraint],
    column2 datatype,
    ...
);
```

Example:

```
SQL> CREATE TABLE employees (
  2      emp_id NUMBER PRIMARY KEY,
  3      emp_name VARCHAR2(50),
  4      salary NUMBER(10,2)
  5  );

Table created.
```

Step 5: Insert data into the table

Description: INSERT INTO adds one row at a time into the table.

Syntax:

```
INSERT INTO table_name VALUES (value1, value2, ...);
```

Example:

```
SQL> INSERT INTO employees VALUES (1, 'John Doe', 50000);
1 row created.

SQL> INSERT INTO employees VALUES (2, 'Jane Smith', 60000);
1 row created.
```

Step 6: Verify the data

Description: SELECT displays the stored rows, which verifies that the tablespace and quota work.

Syntax:

```
SELECT * FROM table_name;
```

Example:

```
SQL> SELECT * FROM employees;

    EMP_ID EMP_NAME                  SALARY
---------- -------------------- ----------
         1 John Doe                  50000
         2 Jane Smith                60000
```

Practical No. 3

Aim: Create a new role named manager with same privileges. Grant this role to new user. Then change the password of newly created user.

Step 1: Login as DBA (SYSTEM user)

Description: Only a DBA user has the rights to create users, tablespaces and change the database, so first connect as SYSTEM in SQL*Plus.

Credentials: Username: system, Password: the password set during Oracle installation.

Syntax:

```
CONNECT username/password;
```

Example:

```
SQL> connect system/<password>;
Connected.
```

Step 2: Create a role

Description: A role is a named group of privileges. It makes granting many privileges to many users easy.

Syntax:

```
CREATE ROLE role_name;
```

Example:

```
SQL> create role manager;

Role created.
```

Step 3: Grant privileges to the role

Description: Give the same privileges (CREATE TABLE, CREATE VIEW) to the role manager.

Syntax:

```
GRANT privilege1, privilege2 TO role_name;
```

Example:

```
SQL> grant create table, create view to manager;

Grant succeeded.
```

Step 4: Grant the role to the user

Description: The user Prerna gets all privileges of the role manager.

Syntax:

```
GRANT role_name TO username;
```

Example:

```
SQL> grant manager to Prerna;

Grant succeeded.
```

Step 5: Change the password of the user

Description: Connect as the user and use the PASSWORD command (asks old and new password). A DBA can also use ALTER USER ... IDENTIFIED BY.

Credentials: Username: Prerna, Password: prerna123 (old password)

Syntax:

```
PASSWORD
  or
ALTER USER username IDENTIFIED BY new_password;
```

Example:

```
SQL> password
Changing password for PRERNA
Old password:
New password:
Retype new password:
Password changed
```

Practical No. 4

Aim: Revoke the privileges from role manager. Revoke role from user then delete that user.

Note: *If the user already owns objects (for example the employees table from Practical 2), DROP USER gives an error; then use DROP USER Prerna CASCADE; to delete the user with its objects. Please test and run it yourself before writing.*

Step 1: Login as DBA (SYSTEM user)

Description: Only a DBA user has the rights to create users, tablespaces and change the database, so first connect as SYSTEM in SQL*Plus.

Credentials: Username: system, Password: the password set during Oracle installation.

Syntax:

```
CONNECT username/password;
```

Example:

```
SQL> connect system/<password>;
Connected.
```

Step 2: Revoke privileges from the role

Description: REVOKE takes back the privileges given to the role manager.

Syntax:

```
REVOKE privilege1, privilege2 FROM role_name;
```

Example:

```
SQL> Revoke create table, create view from manager;

Revoke succeeded.
```

Step 3: Revoke the role from the user

Description: The user Prerna no longer gets the privileges of role manager.

Syntax:

```
REVOKE role_name FROM username;
```

Example:

```
SQL> Revoke manager from Prerna;

Revoke succeeded.
```

Step 4: Delete the user

Description: DROP USER removes the user from the database. CASCADE also removes all objects owned by the user.

Syntax:

```
DROP USER username [CASCADE];
```

Example:

```
SQL> drop user Prerna;

User dropped.
```

Practical No. 5

Aim: Create a tablespace named “P_student” with file name “Stu_datafile.dbf” having size of 10mb.

Note: *The answer is corrected here: the tablespace name is changed from Std_student (answer sheet) to P_student as given in the practical list. Please test and run it yourself before writing.*

Step 1: Login as DBA (SYSTEM user)

Description: Only a DBA user has the rights to create users, tablespaces and change the database, so first connect as SYSTEM in SQL*Plus.

Credentials: Username: system, Password: the password set during Oracle installation.

Syntax:

```
CONNECT username/password;
```

Example:

```
SQL> connect system/<password>;
Connected.
```

Step 2: Grant CREATE TABLESPACE privilege (if a normal user creates it)

Description: Creating a tablespace needs the CREATE TABLESPACE system privilege.

Syntax:

```
GRANT CREATE TABLESPACE TO username;
```

Example:

```
SQL> GRANT CREATE TABLESPACE TO Prerna;

Grant succeeded.
```

Step 3: Create the tablespace

Description: A tablespace is a logical storage unit; its data is stored in one or more physical datafiles (.dbf).

Syntax:

```
CREATE TABLESPACE tablespace_name
    DATAFILE 'file_name.dbf'
    SIZE n M;
```

Example:

```
SQL> CREATE TABLESPACE P_student
  2  DATAFILE 'Stu_datafile.dbf'
  3  SIZE 10M;

Tablespace created.
```

Practical No. 6

Aim: Create a tablespace named “ts_mydemo” with two datafile named “ts_mydemo01.dbf” & “ts_mydemo02.dbf” having size of 50mb and 64mb respectively. Make the tablespace auto extendable and limit auto extending to size of 2048mb. In auto extending, next extend should of 32mb.

Note: *The answer is corrected here: the datafile names are changed from mydemo01.dbf / mydemo02.dbf (answer sheet) to ts_mydemo01.dbf / ts_mydemo02.dbf as given in the practical list. Please test and run it yourself before writing.*

Step 1: Login as DBA (SYSTEM user)

Description: Only a DBA user has the rights to create users, tablespaces and change the database, so first connect as SYSTEM in SQL*Plus.

Credentials: Username: system, Password: the password set during Oracle installation.

Syntax:

```
CONNECT username/password;
```

Example:

```
SQL> connect system/<password>;
Connected.
```

Step 2: Create the tablespace with two auto-extendable datafiles

Description: Datafiles are separated by commas. AUTOEXTEND ON lets the file grow automatically; NEXT gives the size of each increase and MAXSIZE the upper limit.

Syntax:

```
CREATE TABLESPACE tablespace_name
    DATAFILE 'file1.dbf' SIZE n M
        AUTOEXTEND ON NEXT n M MAXSIZE n M,
    'file2.dbf' SIZE n M
        AUTOEXTEND ON NEXT n M MAXSIZE n M;
```

Example:

```
SQL> CREATE TABLESPACE ts_mydemo
  2  DATAFILE 'ts_mydemo01.dbf' SIZE 50M
     AUTOEXTEND ON NEXT 32M
     MAXSIZE 2048M,
  3  'ts_mydemo02.dbf' SIZE 64M
     AUTOEXTEND ON NEXT 32M
     MAXSIZE 2048M;

Tablespace created.
```

Practical No. 7

Aim: Alter the last datafile “ts_mydemo01.dbf” so that its size will automatically extend by 5mb & limit its size to 500mb.

Note: *The datafile name/path is corrected to TS_MYDEMO01.DBF to match Practical 6. The full path depends on your Oracle installation. Please test and run it yourself before writing.*

Step 1: Login as DBA (SYSTEM user)

Description: Only a DBA user has the rights to create users, tablespaces and change the database, so first connect as SYSTEM in SQL*Plus.

Credentials: Username: system, Password: the password set during Oracle installation.

Syntax:

```
CONNECT username/password;
```

Example:

```
SQL> connect system/<password>;
Connected.
```

Step 2: Find the full path of the datafile

Description: The data dictionary view dba_data_files shows the full path of every datafile of a tablespace.

Syntax:

```
SELECT file_id, file_name
FROM dba_data_files
WHERE tablespace_name = 'TABLESPACE_NAME';
```

Example:

```
SQL> SELECT file_id, file_name
  2  FROM dba_data_files
  3  WHERE tablespace_name = 'TS_MYDEMO';

   FILE_ID FILE_NAME
---------- ------------------------------------------------------------
        10 C:\ORACLEXE\APP\ORACLE\PRODUCT\11.2.0\SERVER\DATABASE\TS_MYDEMO01.DBF
        11 C:\ORACLEXE\APP\ORACLE\PRODUCT\11.2.0\SERVER\DATABASE\TS_MYDEMO02.DBF
```

Step 3: Alter the datafile

Description: ALTER DATABASE DATAFILE changes the auto-extend setting of an existing datafile.

Syntax:

```
ALTER DATABASE DATAFILE 'full_path_of_file'
    AUTOEXTEND ON NEXT n M
    MAXSIZE n M;
```

Example:

```
SQL> ALTER DATABASE
  2  DATAFILE
'C:\ORACLEXE\APP\ORACLE\PRODUCT\11.2.0\SERVER\DATABASE\TS_MYDEMO01.DBF'
  3  AUTOEXTEND ON
  4  NEXT 5M
  5  MAXSIZE 500M;

Database altered.
```

Practical No. 8

Aim: Create temporary table space with size of 5mb which will be automatically increase on demand.

Note: *The answer is corrected here: "temp file" and "auto extend" are written as TEMPFILE and AUTOEXTEND (correct Oracle syntax). Please test and run it yourself before writing.*

Step 1: Login as DBA (SYSTEM user)

Description: Only a DBA user has the rights to create users, tablespaces and change the database, so first connect as SYSTEM in SQL*Plus.

Credentials: Username: system, Password: the password set during Oracle installation.

Syntax:

```
CONNECT username/password;
```

Example:

```
SQL> connect system/<password>;
Connected.
```

Step 2: Create the temporary tablespace

Description: A temporary tablespace stores temporary data during sorting (ORDER BY, GROUP BY, index creation). It uses TEMPFILE instead of DATAFILE; AUTOEXTEND ON increases its size on demand.

Syntax:

```
CREATE TEMPORARY TABLESPACE tablespace_name
    TEMPFILE 'file_name.dbf' SIZE n M
    AUTOEXTEND ON;
```

Example:

```
SQL> create temporary tablespace temp_demo
  2  tempfile 'temp01.dbf' size 5m autoextend on;

Tablespace created.
```

Practical No. 9

Aim: Add new Redo log file of size 10mb to existing database.

Step 1: Login as DBA (SYSTEM user)

Description: Only a DBA user has the rights to create users, tablespaces and change the database, so first connect as SYSTEM in SQL*Plus.

Credentials: Username: system, Password: the password set during Oracle installation.

Syntax:

```
CONNECT username/password;
```

Example:

```
SQL> connect system/<password>;
Connected.
```

Step 2: Add the redo log file

Description: Redo log files record all changes made to the database and are used for recovery. ALTER DATABASE ADD LOGFILE adds a new redo log file (group).

Syntax:

```
ALTER DATABASE ADD LOGFILE 'path/file_name' SIZE n M;
```

Example:

```
SQL> alter database add logfile 'c:/program files/oracle/log5.ora' size 10m;

Database altered.
```

Practical No. 10

Aim: Display the list of all Redo log files and Control files.

Note: *The output of v$controlfile was not given in the answer sheet; a typical Oracle XE output is added here. Please test and run it yourself before writing.*

Step 1: Login as DBA (SYSTEM user)

Description: Only a DBA user has the rights to create users, tablespaces and change the database, so first connect as SYSTEM in SQL*Plus.

Credentials: Username: system, Password: the password set during Oracle installation.

Syntax:

```
CONNECT username/password;
```

Example:

```
SQL> connect system/<password>;
Connected.
```

Step 2: Display the redo log files

Description: The dynamic performance view v$logfile lists every redo log file (member) with its group and status.

Syntax:

```
SELECT * FROM v$logfile;
```

Example:

```
SQL> select * from v$logfile;

    GROUP# STATUS  TYPE    MEMBER
---------- ------- ------- ------------------------------------------------
         2         ONLINE  C:\ORACLEXE\APP\ORACLE\FLASH_RECOVERY_AREA\XE\
                           ONLINELOG\O1_MF_2_MM8H1MLH_.LOG
         1 STALE   ONLINE  C:\ORACLEXE\APP\ORACLE\FLASH_RECOVERY_AREA\XE\
                           ONLINELOG\O1_MF_1_MM8H1MHF_.LOG
```

Step 3: Display the control files

Description: The view v$controlfile lists the control files, which store the structure of the database (datafile and redo log names, checkpoints).

Syntax:

```
SELECT * FROM v$controlfile;
```

Example:

```
SQL> select * from v$controlfile;

STATUS  NAME                                       IS_ BLOCK_SIZE
FILE_SIZE_BLKS
------- ---------------------------------------- --- ---------- -------------
-
        C:\ORACLEXE\APP\ORACLE\ORADATA\XE\CONTROL.DBF NO       16384
594
```

Practical No. 11

Aim: Display the list, datafiles, tablespace names & size in MB.

Step 1: Login as DBA (SYSTEM user)

Description: Only a DBA user has the rights to create users, tablespaces and change the database, so first connect as SYSTEM in SQL*Plus.

Credentials: Username: system, Password: the password set during Oracle installation.

Syntax:

```
CONNECT username/password;
```

Example:

```
SQL> connect system/<password>;
Connected.
```

Step 2: Set the output format

Description: SQL*Plus commands make the report readable: LINESIZE/PAGESIZE set the page, BREAK groups rows by tablespace, COL sets column width.

Syntax:

```
SET LINESIZE n
SET PAGESIZE n
BREAK ON column SKIP 1
COL column FORMAT aN
```

Example:

```
SQL> clear breaks;
SQL> set linesize 130
SQL> set pagesize 60
SQL> break on tablespace_name skip 1
SQL> col tablespace_name format a15
SQL> col file_name format a50
```

Step 3: Display datafiles with tablespace name and size

Description: dba_data_files stores every datafile with its tablespace and size in bytes; bytes/1024000 gives the size in MB.

Syntax:

```
SELECT file_name, tablespace_name, ROUND(bytes/1024000) mb
FROM dba_data_files
ORDER BY 1;
```

Example:

```
SQL> select file_name,tablespace_name, ROUND(bytes/1024000) mb
  2  from dba_data_files order by 1;

FILE_NAME                          TABLESPACE_NAME   MB
---------------------------------- --------------- ----
C:\ORACLEXE\ORADATA\DATAFILE.DBF      TESTING          10
C:\ORACLEXE\ORADATA\TS_MYDEMO01.DBF   TS_MYDEMO        51
C:\ORACLEXE\ORADATA\TS_MYDEMO02.DBF                    66
C:\ORACLEXE\ORADATA\XE\DATAFILE.DBF   DBA_TABLESPACE   10
C:\ORACLEXE\ORADATA\XE\SYSAUX.DBF     SYSAUX          451
C:\ORACLEXE\ORADATA\XE\SYSTEM.DBF     SYSTEM          348
C:\ORACLEXE\ORADATA\XE\UNDO.DBF       UNDO             92
C:\ORACLEXE\ORADATA\XE\USERS.DBF      USERS           102

8 rows selected.
```

Practical No. 12

Aim: Create rollback segment in separate tablespace with proper storage option.

Note: *In Oracle 10g/11g, the database uses automatic undo management, so CREATE ROLLBACK SEGMENT may give error ORA-30019. It works only when UNDO_MANAGEMENT = MANUAL. Please test and run it yourself before writing.*

Step 1: Login as DBA (SYSTEM user)

Description: Only a DBA user has the rights to create users, tablespaces and change the database, so first connect as SYSTEM in SQL*Plus.

Credentials: Username: system, Password: the password set during Oracle installation.

Syntax:

```
CONNECT username/password;
```

Example:

```
SQL> connect system/<password>;
Connected.
```

Step 2: Create a separate tablespace for rollback segments

Description: DEFAULT STORAGE gives the storage option: INITIAL (first extent), NEXT (next extent) and MAXEXTENTS (maximum number of extents).

Syntax:

```
CREATE TABLESPACE tablespace_name
    DATAFILE 'file.dbf' SIZE n M REUSE
    DEFAULT STORAGE (INITIAL n K NEXT n K MAXEXTENTS n);
```

Example:

```
SQL> Create tablespace small_rollbacks
  2  datafile 'c:program files/oracle/small_rollbacks.dbf' size 20m reuse
  3* default storage(initial 250k next 250k maxextents 20)

Tablespace created.
```

Step 3: Suppress undo errors for the session

Description: Allows rollback segment statements to run without undo-mode errors in this session.

Syntax:

```
ALTER SESSION SET undo_suppress_errors = TRUE;
```

Example:

```
SQL> alter session set undo_suppress_errors=true;

Session altered.
```

Step 4: Create the rollback segment

Description: A rollback segment stores the old values of data (before changes) so that a transaction can be rolled back.

Syntax:

```
CREATE ROLLBACK SEGMENT segment_name
    TABLESPACE tablespace_name
    STORAGE (INITIAL n K NEXT n K MAXEXTENTS n);
```

Example:

```
SQL> create rollback segment small_rollbacks
  2  tablespace small_rollbacks
  3  storage(initial 250k next 250k maxextents 20);

Rollback segment created.
```

Practical No. 13

Aim: Create a 3 new abstract datatype as given below: 1. NAME_TYPE(F_NAME, M_NAME, L_NAME) 2. POST_TYPE(MAIN, SFX) 3. ADDR_TYPE(STREET, CITY, STATE, POSTCODE)

Note: *The answer is corrected here: the attribute m_init (answer sheet) is renamed to m_name and add_type is renamed to addr_type, as given in the practical list. Please test and run it yourself before writing.*

Step 1: Create NAME_TYPE

Description: An abstract datatype (object type) groups related attributes into one user-defined type that can be reused in tables.

Syntax:

```
CREATE TYPE type_name AS OBJECT (
    attribute1 datatype,
    attribute2 datatype,
    ...
);
/
```

Example:

```
SQL> create type name_type as object(f_name varchar(15),
  2  m_name char(1),
  3  l_name varchar2(20));
  4  /

Type created.
```

Step 2: Create POST_TYPE

Description: POST_TYPE stores the main postal code and its suffix.

Syntax:

```
CREATE TYPE type_name AS OBJECT (attribute datatype, ...);
/
```

Example:

```
SQL> create type post_type as object(main integer, sfx integer);
  2  /

Type created.
```

Step 3: Create ADDR_TYPE (nested type)

Description: ADDR_TYPE uses POST_TYPE as the datatype of postcode — an object type inside another object type.

Syntax:

```
CREATE TYPE type_name AS OBJECT (
    attribute1 datatype,
    attribute2 other_object_type
);
/
```

Example:

```
SQL> create type addr_type as object(
  2  street varchar2(35),
  3  city varchar(15), state char(2), postcode post_type);
  4  /

Type created.
```

Practical No. 14

Aim: Create a table stud_info having column name, rollno, phy, maths, comp, total and percentage. Write a trigger to automatically calculate students total marks and percentage of students while insertion.

Note: *The answer is corrected here: per is changed from NUMBER(3,2) to NUMBER(5,2), because NUMBER(3,2) can store only up to 9.99 and gives an error for real percentages like 83.33. Also "endtotper;" is written as "end totper;". Please test and run it yourself before writing.*

Step 1: Create the table

Description: The table stores marks of three subjects; total and per are filled automatically by the trigger.

Syntax:

```
CREATE TABLE table_name (column datatype, ...);
```

Example:

```
SQL> create table studinfo
  2  (name varchar2(20),
  3  rollno number(3),
  4  phy number(3),
  5  math number(3),
  6  comp number(3),
  7  total number(5),
  8  per number(5,2));

Table created.
```

Step 2: Create the trigger

Description: A BEFORE INSERT ... FOR EACH ROW trigger runs before every new row is inserted. :new refers to the values of the new row, so total and per are calculated automatically.

Syntax:

```
CREATE OR REPLACE TRIGGER trigger_name
BEFORE INSERT ON table_name
FOR EACH ROW
BEGIN
    :new.column := expression;
END trigger_name;
/
```

Example:

```
SQL> create or replace trigger totper
  2  before insert on studinfo
  3  for each row
  4  begin
  5  :new.total:=:new.math+:new.phy+:new.comp;
  6  :new.per:=(:new.total/300)*100;
  7  end totper;
  8  /

Trigger created.
```

Step 3: Insert a record (without total and percentage)

Description: Only name, rollno and marks are inserted; the trigger fills total and per.

Syntax:

```
INSERT INTO table_name (col1, col2, ...) VALUES (v1, v2, ...);
```

Example:

```
SQL> insert into studinfo(name,rollno,phy,math,comp)
  2  values('sid',1,7,7,7);

1 row created.
```

Step 4: Display the table

Description: The output shows the total and percentage calculated by the trigger.

Syntax:

```
SELECT * FROM table_name;
```

Example:

```
SQL> select * from studinfo;

NAME      ROLLNO   PHY  MATH  COMP  TOTAL   PER
--------- ------ ----- ----- ----- ------ -----
sid            1     7     7     7     21     7
```

Practical No. 15

Aim: Write a SQL query to display the following data information: a. Name of database b. Database startup time c. Database size information (database size, used space, free space)

Step 1: Login as DBA (SYSTEM user)

Description: Only a DBA user has the rights to create users, tablespaces and change the database, so first connect as SYSTEM in SQL*Plus.

Credentials: Username: system, Password: the password set during Oracle installation.

Syntax:

```
CONNECT username/password;
```

Example:

```
SQL> connect system/<password>;
Connected.
```

Step 2: Display the name of the database

Description: The view v$database contains information about the database, including its name.

Syntax:

```
SELECT name FROM v$database;
```

Example:

```
SQL> select name from v$database;

NAME
---------
ORCL
```

Step 3: Display the database startup time

Description: The view v$instance stores the startup time of the instance; TO_CHAR formats the date.

Syntax:

```
SELECT TO_CHAR(startup_time, 'format') FROM v$instance;
```

Example:

```
SQL> select to_char(startup_time,'hh24:mi dd-mm-yy') "startup time"
  2  from v$instance;

startup time
--------------
11:10 06-10-18
```

Step 4: Display database size, used space and free space

Description: Size = sum of bytes of datafiles (v$datafile), tempfiles (v$tempfile) and redo logs (v$log). Free space = sum of bytes in dba_free_space. Dividing by 1024 converts bytes to KB, MB and GB.

Syntax:

```
SELECT SUM(bytes) ... FROM (v$datafile UNION ALL v$tempfile UNION ALL v$log),
       (SELECT SUM(bytes) FROM dba_free_space) ...;
```

Example:

```
SQL> select round(sum(used.bytes)/1024/1024/1024)|| 'gb' "database size",
  2  round(sum(used.bytes)/1024/1024) || 'mb' "used space",
  3  round(free.p/1024/1024) || 'mb' "free space"
  4  from (select bytes from v$datafile
  5        union all
  6        select bytes from v$tempfile
  7        union all
  8        select bytes from v$log) used,
  9  (select sum(bytes) as p from dba_free_space) free
 10  group by free.p;

database size   used space     free space
--------------- -------------- ------------
1gb             1104mb         186mb
```